

Capítulo 10

Distributed

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

Capítulo 10 — Distributed (treinar em vários dispositivos)

Objetivo de aprendizagem: entender como N processos treinam **o mesmo modelo** ao mesmo tempo. Vamos implementar o **ring all-reduce** do zero, provar que o **DDP** reproduz exatamente o gradiente do batch inteiro, e ver como o **ZeRO** elimina a duplicação de memória que o DDP deixa passar.

Pré-requisitos: Capítulos 1-9. Fecha a Fase III ("Need for Speed").

Arquivos:

- `dist_utils.py` — infraestrutura (rendezvous e rede)
- `allreduce.py` — a primitiva, **implementada à mão** e verificada
- `ddp_train.py` — treino com DDP, com prova de equivalência
- `zero_memory.py` — a conta de memória e o ZeRO medido
- `exercicios.md` — exercícios

Sobre o hardware: esta máquina tem **uma** GPU, então não é possível medir ganho real de velocidade com múltiplas GPUs. O que **é** verificável — e é o conteúdo conceitual do capítulo — roda com **múltiplos processos na CPU** usando o backend `gloo`: a mecânica do all-reduce, a equivalência matemática do DDP e o fatiamento do ZeRO. Tudo neste capítulo foi executado de verdade; o que não foi medido está declarado na Seção 8.

1. A ideia: paralelismo de dados

O modelo cabe numa placa, mas os dados são muitos. A solução mais usada:

1. Todo processo tem uma **cópia completa e idêntica** do modelo.
2. Cada um recebe um **pedaço diferente** do batch.
3. Cada um calcula gradientes sobre o seu pedaço.
4. Um **all-reduce** faz a **média** dos gradientes.
5. Todos aplicam a mesma atualização -> os modelos continuam idênticos.

O passo 4 é o coração de tudo. Sem ele, os modelos divergiriam e você teria N modelos ruins em vez de um bom.

2. All-reduce: todos contribuem, todos recebem

```
t = torch.tensor([float(rank + 1)]) # rank 0 tem 1, rank 1 tem 2, ...
dist.all_reduce(t, op=dist.ReduceOp.SUM)
```

Com 4 processos:

```
[rank 0] tinha 1, depois do all_reduce SUM: 10 (esperado 10) OK
[rank 1] tinha 2, depois do all_reduce SUM: 10 (esperado 10) OK
[rank 2] tinha 3, depois do all_reduce SUM: 10 (esperado 10) OK
[rank 3] tinha 4, depois do all_reduce SUM: 10 (esperado 10) OK
```

Todos terminam com o mesmo valor. Não é "o rank 0 calcula e avisa os outros" — é uma operação coletiva em que cada participante entra com um valor e sai com o resultado combinado. É essa simetria que mantém os modelos sincronizados.

Para gradientes, queremos a **média**, que é **SUM** seguido de divisão:

```
dist.all_reduce(grad, op=dist.ReduceOp.SUM)
grad /= world_size
```

```
[rank 0] gradiente local 0.10 -> medio 0.2500
[rank 1] gradiente local 0.20 -> medio 0.2500
[rank 2] gradiente local 0.30 -> medio 0.2500
[rank 3] gradiente local 0.40 -> medio 0.2500
```

3. Como o all-reduce funciona por dentro: o anel

A implementação ingênua seria: todos mandam para o rank 0, ele soma e devolve. **Não escala.** O rank 0 recebe $(N-1) \times \text{tamanho}$ e devolve $(N-1) \times \text{tamanho}$ — ele vira um gargalo que **piora** conforme N cresce.

A solução usada na prática é o **ring all-reduce**, em que os processos formam um anel e cada um só fala com os vizinhos. Ele tem duas fases, e a chave é **fatiar o tensor em N pedaços**:

Fase 1 — reduce-scatter. Em $N-1$ passos, cada processo passa um pedaço ao vizinho da direita e soma o que recebe da esquerda. Ao final, cada processo tem **um** pedaço completamente somado (pedaços diferentes em cada um).

Fase 2 — all-gather. Em mais $N-1$ passos, esses pedaços já prontos circulam pelo anel até todos terem todos.

O ganho: **cada processo envia e recebe sempre a mesma quantidade** ($\sim 2 \times$ o tamanho do tensor), **independentemente de N** . É por isso que o ring all-reduce escala para centenas de nós.

Implementamos do zero em `allreduce.py`:

```
for passo in range(world_size - 1):
    idx_envio = (rank - passo) % world_size
    idx_recebe = (rank - passo - 1) % world_size
    recebido = torch.zeros_like(pedacos[idx_recebe])
    if rank % 2 == 0: # ordem par/ímpar evita deadlock
        dist.send(pedacos[idx_envio].contiguous(), dst=direita)
        dist.recv(recebido, src=esquerda)
    else:
        dist.recv(recebido, src=esquerda)
```

```
dist.send(pedacos[idx_envio].contiguous(), dst=direita)
pedacos[idx_recebe] += recebido
```

O detalhe do par/ímpar: se todos os processos enviassem primeiro e o buffer do sistema enchesse, todos ficariam bloqueados esperando alguém receber — um **deadlock**. Alternar a ordem entre pares e ímpares garante que sempre há alguém pronto para receber.

E funciona:

```
[rank 0] nosso == PyTorch? True (primeiros 3: nosso [600.0, 604.0, 608.0], pytorch [600.0, 604.0, 608.0])
[rank 1] nosso == PyTorch? True
[rank 2] nosso == PyTorch? True
[rank 3] nosso == PyTorch? True
```

Nossa implementação bate com a do PyTorch — a quarta verificação contra a biblioteca de referência no curso, depois do autograd (Cap. 2), da LayerNorm (Cap. 5) e do AdamW (Cap. 7).

4. Quanto custa comunicar

Medido com 4 processos na mesma máquina:

Elementos	MB	Tempo	Vazão
10 mil	0,04	2,90 ms	14 MB/s
1 milhão	4,0	12,88 ms	310 MB/s
10 milhões	40,0	107,83 ms	371 MB/s

Note que tensores pequenos têm vazão péssima (14 MB/s): o custo fixo por operação domina — o mesmo padrão que vimos na GPU no Capítulo 8. Por isso frameworks **agrupam** gradientes em *buckets* antes de comunicar, em vez de fazer um all-reduce por parâmetro.

E aqui está o limite fundamental do treino distribuído:

A cada passo, **todo o gradiente do modelo atravessa a rede**. Um modelo de 1 bilhão de parâmetros gera **4 GB de gradientes por passo**. Se a comunicação demorar mais que o cálculo, acrescentar máquinas **para de ajudar**. É por isso que existem NVLink e InfiniBand — e por que a topologia da rede é parte do projeto do treino.

5. DDP: o gradiente é o mesmo do batch inteiro?

Esta é a propriedade que justifica tudo. O `DistributedDataParallel` dispara o all-reduce automaticamente durante o `backward()`. A pergunta: o resultado é **exatamente** o mesmo de treinar com

o batch inteiro num processo só?

O `ddp_train.py` compara diretamente — calcula o gradiente de referência com batch 256 num processo, e o gradiente do DDP com 4 processos de 64 cada:

```
batch total 256, dividido em 4 pedacos de 64
[rank 0] diferenca maxima vs batch unico: 1.12e-08 OK
[rank 1] diferenca maxima vs batch unico: 1.12e-08 OK
[rank 2] diferenca maxima vs batch unico: 1.12e-08 OK
[rank 3] diferenca maxima vs batch unico: 1.12e-08 OK
```

São iguais, a menos de arredondamento. O DDP não aproxima nada: ele reproduz a matemática exata do batch inteiro.

E ao longo do treino, os modelos continuam sincronizados:

```
[rank 0] loss final 2.1846 | pesos identicos entre ranks: True
[rank 1] loss final 2.3620 | pesos identicos entre ranks: True
[rank 2] loss final 2.1699 | pesos identicos entre ranks: True
[rank 3] loss final 2.3078 | pesos identicos entre ranks: True
```

Repare que as losses são diferentes! Cada rank imprime a loss do **seu pedaço** do batch. Para registrar a loss real do treino é preciso fazer `all_reduce` dela também — e esquecer disso é um erro comum, que faz o gráfico de treino parecer mais ruidoso do que é.

6. O cuidado que mais causa problema: batch efetivo

Com 4 processos de 64 exemplos cada, o batch **efetivo** é **256**, não 64. Isso muda um hiperparâmetro importante:

- Gradiente de batch maior é **menos ruidoso**.
- Logo, tolera (e costuma pedir) **learning rate maior**.
- Regra prática usual: multiplicar a lr por `√world_size`, ou linearmente com warmup mais longo.

Trocar 1 GPU por 8 **sem tocar na learning rate** é um erro clássico: cada passo fica mais rápido, mas o treino precisa de **mais passos** para chegar ao mesmo lugar — e o ganho evapora. Releia o exercício E5 do Capítulo 7: a melhor learning rate depende do tamanho do batch.

7. ZeRO: o desperdício que o DDP não resolve

O DDP resolve **velocidade**, não **memória**. Cada processo guarda uma cópia completa de tudo. Fazendo a conta, por parâmetro, num treino com AdamW em fp32:

Componente	Bytes/param
------------	-------------

pesos	4
gradientes	4
AdamW: momento m	4
AdamW: momento v	4
TOTAL	16 — em cada processo

Um modelo de 1 bilhão de parâmetros pede **16 GB por GPU**, e esse número **não diminui** por mais GPUs que você acrescente.

O **ZeRO** (*Zero Redundancy Optimizer*) elimina a duplicação em três estágios:

Processos	DDP	ZeRO-1	ZeRO-2	ZeRO-3
1	16,0 GB	16,0 GB	16,0 GB	16,0 GB
4	16,0 GB	10,0 GB	7,0 GB	4,0 GB
8	16,0 GB	9,0 GB	5,5 GB	2,0 GB
64	16,0 GB	8,1 GB	4,2 GB	0,25 GB

- **ZeRO-1** fatia o estado do otimizador (**m**, **v**)
- **ZeRO-2** fatia também os gradientes
- **ZeRO-3** fatia também os pesos, buscados sob demanda durante o forward

Leia a linha de 64 processos: o DDP continua exigindo 16 GB por GPU — o mesmo de rodar sozinho. O ZeRO-3 pede 0,25 GB. É a diferença entre "não cabe" e "cabe folgado".

O preço é comunicação. O ZeRO-3 precisa buscar pesos de outros processos durante o forward e o backward. Troca-se memória por banda — e o estágio certo depende de qual é o seu gargalo.

Medido de verdade

O `zero_memory.py` usa o `ZeroRedundancyOptimizer` do PyTorch (ZeRO-1) num modelo de 1.067.040 parâmetros, com 4 processos:

```
[rank 0] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.23 MB (3.8x menor)
[rank 1] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.11 MB (4.1x menor)
[rank 2] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.10 MB (4.1x menor)
[rank 3] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.10 MB (4.1x menor)
```

Cada processo guarda ~1/4 do estado, exatamente a promessa. (Os pesos e gradientes seguem duplicados — é o que os estágios 2 e 3 atacam.)

8. Um problema real de infraestrutura, e a lição

Este capítulo quase não aconteceu, e a razão vale mais que o código.

Ao rodar os primeiros testes, os processos **travavam para sempre** no `init_process_group` — sem erro, sem mensagem, sem timeout. Só um programa pendurado.

A causa levou um tempo para ser encontrada:

```
hostname: SANCHEZPC1
resolve para: 54.232.189.113 <- um IP PÚBLICO, não um endereço local
```

Mesmo rodando tudo numa máquina só, o `gloo` abre sockets TCP entre os processos, e ele escolhe a interface **resolvendo o hostname da máquina**. Nesta máquina o hostname resolvia para um IP público (por causa de um adaptador virtual), e o Windows recusava a conexão com o erro 10049 — "endereço não é válido no contexto".

A correção é dizer explicitamente qual interface usar:

```
GLLO_SOCKET_IFNAME=Ethernet python allreduce.py
```

E o `dist_utils.py` faz isso **automaticamente**: detecta que o hostname não resolve para um endereço local, procura um adaptador com IP privado e configura a variável.

A lição: em treino distribuído, o modo de falha mais comum não é o erro — é o **travamento silencioso**. Rendezvous que não completa, interface errada, um processo que morreu e deixou os outros esperando no `barrier`. Antes de suspeitar do seu código, verifique a rede. E sempre rode com timeout.

O que não foi possível medir aqui

Sendo explícito, como no Capítulo 9:

Item	Verificado	Como
Mecânica do all-reduce	[OK]	4 processos, CPU, backend <code>gloo</code>
Ring all-reduce do zero	[OK]	comparado com o do PyTorch
Equivalência do DDP	[OK]	gradiente vs batch único: 1,12e-08
Sincronização dos pesos	[OK]	<code>all_gather</code> após 50 passos
ZeRO-1	[OK]	<code>ZeroRedundancyOptimizer</code> , 4 processos
Speedup real com N GPUs	[nao]	a máquina tem uma GPU só
Backend NCCL	[nao]	exige GPU NVIDIA

9. Resumo do capítulo

- **Paralelismo de dados:** cópias idênticas do modelo, pedaços diferentes do batch, gradientes combinados por **all-reduce**.
- **All-reduce** é coletivo: todos contribuem e todos recebem o mesmo resultado.
- O **ring all-reduce** faz cada processo trocar sempre a mesma quantidade de dados, independentemente de N — por isso escala. Implementamos do zero e bate com o PyTorch.
- **Comunicação é o gargalo:** todo o gradiente atravessa a rede a cada passo. Tensores pequenos têm vazão péssima -> frameworks agrupam em *buckets*.
- **DDP reproduz exatamente** o gradiente do batch inteiro (verificado: 1,12e-08), e mantém os pesos idênticos entre processos.
- **Batch efetivo** = batch local × world_size -> **ajuste a learning rate**.
- **ZeRO** elimina a duplicação do estado: com 64 processos, 16 GB por GPU (DDP) contra 0,25 GB (ZeRO-3). Medimos o ZeRO-1: 4,1x menos estado por processo.
- **Falhas em distribuído são silenciosas.** O maior obstáculo aqui foi um hostname que resolvia para um IP público — e o sintoma era o programa pendurado.

O que vem no Capítulo 11

Fecha a Fase III e começa a Fase IV. E é a virada que muda o curso: no **Capítulo 11 — Datasets** trocamos os nomes por **texto de verdade**. O modelo deixa de gerar nomes e passa a gerar prosa — e o nosso benchmark de 1,776 será aposentado, porque a tarefa muda.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 10 (Distributed)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Todos os exercícios rodam na CPU, com múltiplos processos. Não é preciso ter GPU nem mais de uma máquina.

Se algo travar: rode `python dist_utils.py` para diagnosticar a rede, e sempre use timeout ao experimentar (`timeout 60 python ...` no Linux/Git Bash). Travamento silencioso é o modo de falha normal aqui — veja a Seção 8 da apostila.

E1 — Entendendo o coletivo (aquecimento)

Sem rodar:

1. Com 4 processos onde o rank `i` tem o valor `i`, qual o resultado do `all_reduce` com `SUM` em cada rank? E com `MAX`?
2. Qual a diferença entre `all_reduce` e `reduce`? E entre `all_gather` e `gather`?
3. Por que o `broadcast` **não** é suficiente para sincronizar gradientes? (Pense: quem teria o valor correto para transmitir?)

E2 — Escalando o número de processos

Rode `python allreduce.py N` com `N = 2, 4 e 8`.

1. O tempo do all-reduce cresce com `N`? Meça com **três tamanhos** de tensor — 10 mil, 1 milhão e 10 milhões de elementos — antes de responder. A resposta é a mesma nos três?
2. Compare com o que aconteceria no esquema ingênuo (todos -> rank 0 -> todos): esse tempo cresceria **como** em função de `N`?
3. Sua máquina tem quantos núcleos? A partir de qual `N` os processos passam a competir por CPU em vez de somar? (Rode `python dist_utils.py` para ver.)

E3 — Provoque um deadlock (importante)

No `ring_allreduce_manual`, remova a alternância par/ímpar — faça **todos** os processos chamarem `dist.send` antes de `dist.recv`.

1. **Preveja antes de rodar.** A explicação clássica do deadlock em `send/recv` é o **buffer de socket**: o sistema operacional aceita mensagens pequenas e devolve o controle na hora, então só as grandes travariam. Sob essa teoria, o que deveria acontecer com um tensor de 100 elementos? E com um de 5 milhões?

2. Agora **meça**, com os dois tamanhos — e também com um tensor de **um único elemento**. O resultado bate com a sua previsão? (Rode cada caso com timeout; o objetivo aqui é justamente travar o programa.)
3. Se a previsão falhou, o que isso diz sobre aplicar um modelo mental correto *em geral* a uma implementação específica? E sobre um backend que falha **sempre** em vez de falhar só às vezes — isso é um defeito ou um recurso?

E4 — A loss que engana

No `ddp_train.py`, os ranks imprimem losses diferentes (2,18 / 2,36 / 2,17 / 2,31).

1. Por quê, se os pesos são idênticos?
2. Implemente o registro **correto** da loss: faça `all_reduce` dela e divida por `world_size`. Os valores passam a bater entre os ranks?
3. Se você registrasse só a loss do rank 0 num gráfico, o que aconteceria com a aparência da curva?

Solução de referência: `solucoes/e4_loss_correta.py`.

E5 — Batch efetivo e learning rate

Treine o `ddp_train.py` com 1, 2 e 4 processos, mantendo o batch **por processo** fixo.

1. O batch efetivo muda. A loss após 50 passos melhora, piora ou fica igual?
2. Agora ajuste a learning rate por $\sqrt{\text{world_size}}$ e repita. Muda a conclusão?
3. Por que "mais GPUs" pode deixar o treino **mais lento em número de passos** se você não mexer na learning rate?

E6 — Meça o ZeRO nos três estágios

O `zero_memory.py` mede o ZeRO-1 de verdade e calcula os estágios 2 e 3 por aritmética.

1. Confirme a conta do ZeRO-1: com N processos, o estado medido é mesmo $\sim 1/N$?
2. A tabela assume AdamW em fp32. Refaça a conta para **SGD sem momentum** (que não guarda estado). O ZeRO ainda ajuda? Quanto?
3. E se combinássemos com o Capítulo 9 (bf16 nos pesos)? Recalcule a linha de 8 processos.

E7 — Quando distribuir *não* compensa (desafio)

Junte o que você mediu neste capítulo e no Capítulo 8.

1. Estime: para o modelo pequeno do curso (153 mil parâmetros), quanto tempo levaria o all-reduce dos gradientes por passo? Compare com o tempo de cálculo de um passo (12,8 ms na CPU, do

Capítulo 8).

2. Distribuir esse modelo em 4 processos aceleraria ou atrasaria o treino?
3. Formule a regra geral: qual a relação entre **tempo de cálculo** e **tempo de comunicação** que decide se vale a pena distribuir?